

SSD_Partition Usage Reference

1. Partition introduction

1.1. Introduction of SPI-NOR Flash Basic Partition

ALKAID will print the layout of all partitions after make image is completed:

```
MXP count max 30
NOR FLASH HAS USED 0x1000000KB
  IPL: 0x00000000-0x00010000 size:64KB
  IPL_CUST: 0x00010000-0x00020000 size:64KB
  MXPT: 0x00020000-0x00030000 size:64KB
  UBOOT: 0x00030000-0x0004F000 size:124KB
  UBOOT_ENV: 0x0004F000-0x00050000 size:4KB
  BOOT: 0x00000000-0x00050000 size:320KB
  KERNEL: 0x00050000-0x00250000 size:2048KB
  rootfs: 0x00250000-0x00650000 size:4096KB
  miservice: 0x00650000-0x00950000 size:3072KB
  customer: 0x00950000-0x01000000 size:6848KB
Available MXP record count:10
```

- IPL

After the CPU is powered on, the first thing to run is the rom code. As the name implies, the code is stored in a special ROM and is read-only. After the ROM code runs, read the location of the NOR Flash 0 address, which is the location where the IPL file is stored. The main function of IPL is to do some basic hardware initialization, such as setting the current DDR parameters, and GPIO/IIC related.

- IPL_CUST

IPL initializes the common hardware modules. IPL_CUST will initialize the executable binary files of customized board hardware according to the actual situation of the current board, such as customized GPIO pins, IIC configuration.

- MXPT

Partition configuration related binaries.

- UBOOT

UBOOT binary file storage partition.

- UBOOT_ENV

The environment variable of UBOOT is stored in the partition.

- logo

It will be used on NVR devices and store the configuration related to the boot logo.

It is not recommended to change the contents of the BOOT partition of SPI-NOR. If there are any places that do not conform to the release of the public board, the system may fail to start.

BOOT is in the position of mtd block0 of linux, the corresponding device node /dev/mtdblock0.

The partitions after the BOOT partition are collectively referred to as the SYS partition. The SYS partition can be modified. According to the actual usage, it can be roughly divided into four categories:

- KERNEL

The binary file that holds the kernel.

- ROOTFS

As the title.

- miservice

This is a partition defined by the public board. It is used to store the mi library and some configuration files. The file system defaults to jffs2.

- customer

User-defined partitions.

1.2. Introduction to the basic partition of SPI-NAND Flash

Layout diagram example:

```
BOOT: 0x140000(CIS),0x20000@0x140000(IPL0)5,0x20000(IPL_CUST0)2,0x20000(IPL_CUST1)2,0x4
SYS: 0x500000(KERNEL),0x500000(RECOVERY),0x600000(rootfs),-(UBI)
FLASH HAS USED 0x8000000KB
ChkSum      : 1043
SpareByteCnt : 64
PageByteCnt  : 2048
BlkPageCnt   : 64
BlkCnt       : 1024
PartCnt      : 11
UnitByteCnt  : 8
Checksum ok!!
IDX:      StartBlk:      BlkCnt:      BackupBlkCnt:      PartType:
0:      0, (0000000000)  10, (0x00140000)  0, (0000000000)  0x0020, (CIS)
1:      10, (0x00140000)  1, (0x00020000)  5, (0x000A0000)  0x0003, (IPL)
2:      16, (0x00200000)  1, (0x00020000)  2, (0x00040000)  0x0001, (IPL_CUST)
3:      19, (0x00260000)  1, (0x00020000)  2, (0x00040000)  0x0001, (IPL_CUST)
4:      22, (0x002C0000)  2, (0x00040000)  1, (0x00020000)  0x0006, (UBOOT)
5:      25, (0x00320000)  2, (0x00040000)  1, (0x00020000)  0x0006, (UBOOT)
6:      28, (0x00380000)  2, (0x00040000)  0, (0000000000)  0x000D, (ENV)
7:      30, (0x003C0000)  40, (0x00500000)  0, (0000000000)  0x0012, (KERNEL)
8:      70, (0x008C0000)  40, (0x00500000)  0, (0000000000)  0x0009, (RECOVERY)
9:      110, (0x00DC0000)  48, (0x00600000)  0, (0000000000)  0x000F, (ROOTFS)
10:     158, (0x013C0000)  866, (0x06C40000)  0, (0000000000)  0x0021, (UBI)
11: records in
```

- CIS

The unique partition of SPI-NAND is stored at the address of flash 0. One part is spinand info, which saves the basic information of spinand, which will change according to the type of spinand, such as block page count, block count, etc.; the other part is partinfo, Save partition information, these information are read by rom code, rom code can run the whole system by reading the correct spinand parameters and partition information to obtain the position of IPL.

- IPL

IPL分区的作用与SPI-NOR分区一样，但是从上图可知，IPL在spi-nand上保存了六份，每个block一份，目的是为了防止坏块而做的备份。

- IPL_CUS

作用同SPI-NOR，有IPL_CUS0和IPL_CUS1两个分区,每个分区中各三份IPL_CUS的data。

- UBOOT

UBOOT的二进制文件存放分区,会有两个分区，每个分区中一份data。

- ENV

UBOOT的环境变量存放分区。

- LOGO

在NVR设备上会使用，存放的是开机logo相关的配置。

以上是BOOT相关的分区信息，这部分无法任意修改。

- KERNEL

存放内核的二进制文件。

- ROOTFS

如题。

- UBI

UBI的内容在上图分区表中不会显示出来，UBI中会创建多个ubifs格式的子分区，用户可以根据需要自行创建。Spinand的miservice分区就是放在UBI中。

1.3. MISERVICE分区内容

在MISERVICE分区的内容被mount到了根目录/config/下面，下表中加粗字体的文件是必须保持路径一致的，否则有可能会導致系统无法启动。

└─ config

└─ **board.ini** ---->保存当前板子的信息

└─ **config_tool** ---->mi sys初始化mmap内存的应用

└─ **dump_config -> config_tool** ---->dump config 应用，**config_tool**的symbol link。

└─ **dump_mmap -> config_tool** ---->dump mmap应用，**config_tool**的symbol link。

└─ **fbdev.ini** ---->Framebuffer 配置，若无fbdev需求可删除。

└─ **iqfile**

- | |— imx307_iqfile.bin
- | |— iqfile0.bin -> imx307_iqfile.bin ---->Sensor IQ相关的bin
- | |— iqfile1.bin -> imx307_iqfile.bin
- | |— iqfile2.bin -> imx307_iqfile.bin
- | |— iqfile3.bin -> imx307_iqfile.bin
- | |— isp_api.xml
- |— mmap.ini ---->Mmap
- |— model ---->模块相关配置，可以删除。
- | |— Customer_1.ini
- | |— Customer_2.ini
- | |— Customer_auto_test.ini
- |— modules ---->系统KO存放路径。
- | |— 4.9.84
- | |— cifs.ko
-
- |— pq ---->NVR显示PQ相关。
- | |— Bandwidth_RegTable.bin
- | |— Main.bin
- | |— Main_Ex.bin
- |— terminfo ---->Console显示相关。
- | |— v
- | |— vt100

.....

└─ **venc_fw** ---->Encoder的firmware

└─ **chagall.bin**

2. 分区变更

2.1. 分区变更前注意事项

无论是spi-nor flash上的分区，还是spi-nand flash上的分区。都可归为两类：

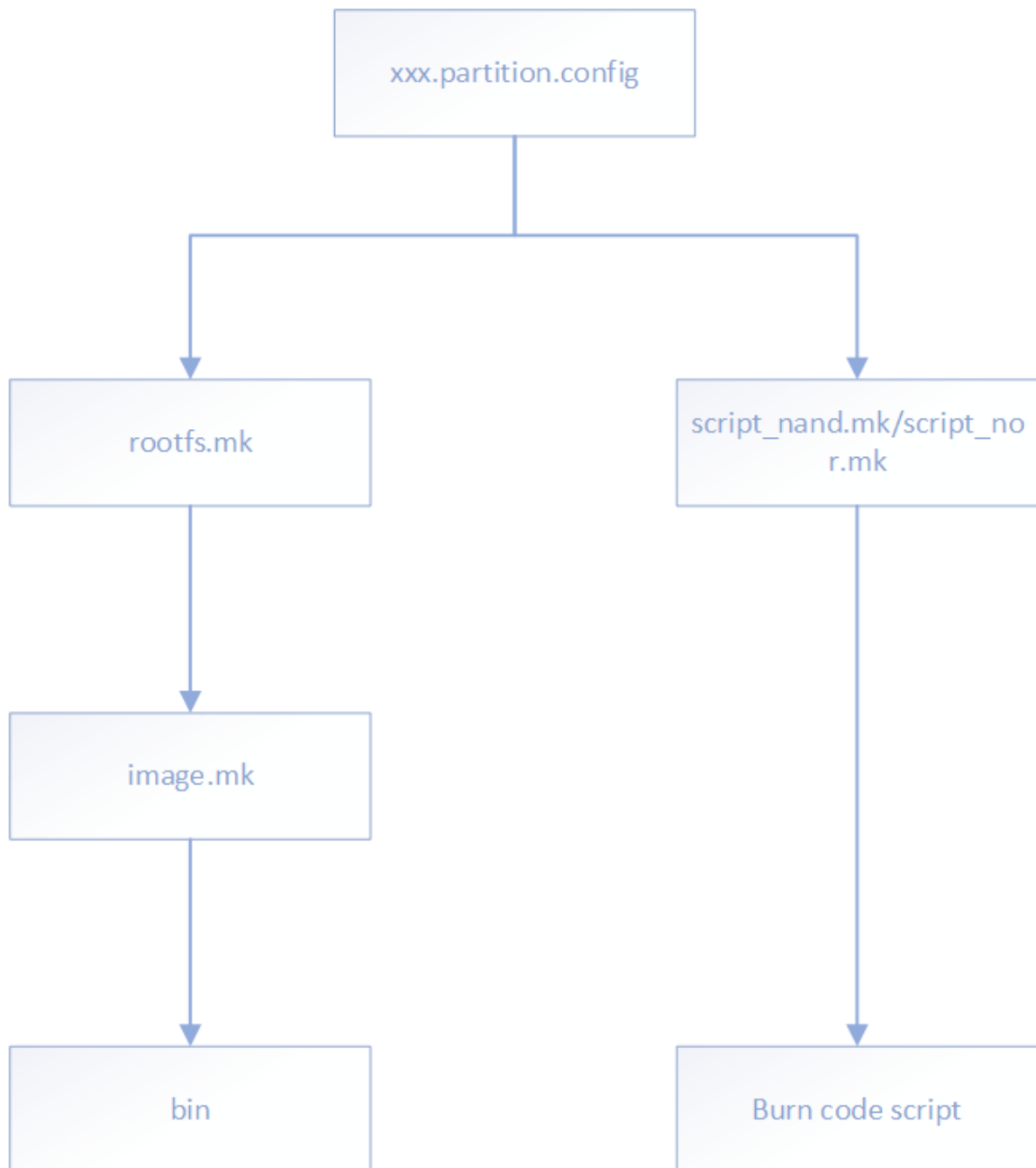
- 不建议改动的boot
- 可以变动的sys

本章介绍的分区变更都是在sys部分上，包括如何配置脚本变更分区顺序，如何打包分区，如何生成烧录脚本，若用户有一套打包和烧录的流程，请忽略本章。注意：在miservice分区中加粗字体标注的文件需要放到指定的路径下系统才能运行，所以在制作分区时务必在根目录下创建/**config/**，并把相关的文件放到此路径下。

2.2. 分区变更的脚本代码架构介绍

分区变更分为分区制作/打包和分区烧录脚本制作这两部分。

下面流程图的两个分支，左边走的是分区制作、打包流程，右边走的是分区烧录脚本制作的流程，分区烧录脚本仅支持tftp升级脚本，暂不支持usb升级脚本制作。



- **xxx.partition.config :**

这是总的分区配置信息脚本，所有与分区调整相关的代码都在这里实现，文件有很多份，每个不同的chip、spinand分区、spinor分区都会有各自的分区config文件。

通过build config找到该项目使用的分区config文件，例如某项目使用的build config：

```
project/configs/ipc/$(CHIP)/nor.glibc-squashfs.009a.64.qfn88
```

此配置脚本中，变量IMAGE_CONFIG可以找到对应使用的分区配置脚本。

此项目的分区配置脚本存放的路径在：

```
project/mage/configs/$(CHIP)/nor.squashfs.partition.config
```

可以看出，这是spinor flash分区配置脚本文件。

在分区config文件中用变量定义了分区BOOT的部分和SYS的部分：

```
Spinor :

mxp$(BOOTTAB) = "xxx"

mxp$(SYSTAB) = "xxx"

Spinand:

cis$(BOOTTAB) = xxx

cis $(SYSTAB) = xxx
```

在config脚本文件中，每个分区由一组变量表示，以上mxp和cis是特殊的分区变量，存放的是一组规定的分区配置(raw data)，特殊的分区由特别的指令生成，详细介绍请参见[特殊分区处理](#) [#jump]。

- **rootfs.mk:**

这是ROOTFS和miservice分区制作脚本，项目使用的是：

```
project/mage/configs/$(CHIP)/rootfs.mk
```

- **script_nand.mk/script_nor.mk:**

这是制作spi-nand和spi-nor烧录脚本的脚本，使用的是：

```
project/mage/configs/$(CHIP)/script_nor.mk

project/mage/configs/$(CHIP)/script_nand.mk
```

- **image.mk:**

分区打包脚本，路径在project/mage/image.mk。

.....

2.3. 分区位置变更

2.3.1. Spinor flash

改变mxp\$(SYSTAB)中所定义的分区前后位置即可，语法格式与linux uboot中使用的mtdpart是一样的。

```
mxp$(SYSTAB) = "$(kernel$(PATSIZE))(KERNEL),$(rootfs$(PATSIZE))
(rootfs),$(miservice$(PATSIZE))(miservice),$(customer$(PATSIZE))
(customer)"
```

其中kernel和rootfs的分区信息由逗号','隔开。

2.3.2. Spinand flash

改变cis\$(SYSTAB)中所定义的分区前后位置即可，语法格式与linux uboot中使用的mtdpart是一样的，这个变量的值也会保存到uboot中的环境变量中。

```
cis$(SYSTAB) = $(kernel$(MTDPART)),$(rootfs$(MTDPART)),-(UBI)
```

其中kernel和rootfs的分区信息由逗号','隔开。

2.4. 增加修改删除jffs2分区

在spinor的flash上一般用jffs2作为可读写的分区的文件系统，如下定义了customer分区的相关配置：

```
customer$(RESOURCE) = $(OUTPUTDIR)/customer
customer$(FSTYPE)    = jffs2
customer$(PATSIZE)   = 0x6B0000
customer$(MOUNTTG)   = /customer
customer$(MOUNTPT)   = mtd:customer
customer$(OPTIONS)   = rw
```

- customer\$(RESOURCE)

表示分区的资源文件放置的位置，`rootfs.mk`和`image.mk`分别会往此路径中添加必须的文件，以及把文件夹打包成bin文件。

- `customer$(FSTYPE)`

打包的文件系统类型。

- `customer$(PATSIZE)`

分区大小。

- `customer$(MOUNTTG)`、`customer$(MOUNTPT)`、`customer$(OPTIONS)`

分区在板子起来后需要mount的路径以及参数。`rootfs.mk`中会处理这些参数。

2.4.1. 修改

若要修改分区，请改动以上变量值。

2.4.2. 增加分区

1. 假设增加一个分区名为`aaa`，然后配置其大小等信息。

```
aaa$(RESOUCE)    = $(OUTPUTDIR)/aaa

aaa$(FSTYPE)     = jffs2

aaa$(PATSIZE)    = 0x6B0000

aaa$(MOUNTTG)    = /aaa

aaa$(MOUNTPT)    = mtd:aaa

aaa$(OPTIONS)    = rw
```

2. 分区相关的变量配置完成之后，在`mvp$(SYSTAB) = "xxx"`中指定其放置的位置。
3. 在变量`"IMAGE_LIST"`后追加`"aaa"`。
4. 在表示需要mount的节点的变量`"USR_MOUNT_BLOCKS"`后追加`"aaa"`。

2.4.3. 删除分区

按照增加分区的操作，反过来执行。

2.4.4. 烧录脚本

系统在 `script_nor.mk` 中会根据分区的配置自动生成。

2.5. 增加修改删除squashfs分区

Squashfs的分区变更与jiffs2分区变更大同小异，主要在`xxx$(FSTYPE)`上有不同，除去`rootfs`这个比较特别的分区之外，`miservice`分区制作成squashfs：

```
miservice$(RESOUC) = $(OUTPUTDIR)/tvconfig/config
miservice$(FSTYPE) = squashfs
miservice$(PATSIZE) = 0x180000
miservice$(MOUNTTG) = /config
miservice$(MOUNTPT) = /dev/mtdblock3
miservice$(OPTIONS) = ro
```

注意：若用的是spinand的分区配置脚本，那么分区放在 `cis$(SYSTAB) = xxx` 中。

与jiffs2分区一样，系统在`script_nor.mk/script_nand.mk`中会根据分区的配置自动生成烧录脚本。

2.6. 增加修改删除UBIFS分区

UBIFS分区一般应用在spinand上作为可读写的分区文件系统。

BIFS的所有的分区都属于UBI的mtd block中的一个子分区。一个普通的UBI分区变量设置如下：

```
miservice$(RESOUC) = $(OUTPUTDIR)/miservice/config
miservice$(FSTYPE) = ubifs
miservice$(PATSIZE) = 0xA00000
miservice$(MOUNTTG) = /config
```

```
miservice$(MOUNTPT) = ubi0:miservice

miservice$(OPTIONS) = rw
```

与jffs2分区变更方法不同的是，ubifs分区不需要添加mtdpart信息，也就是无需在cis\$(SYSTAB) = xxx中添加分区位置。

2.7. 特殊分区处理

Kernel、uboot、IPL、logo等属于特殊分区没有特别的文件系统，无法在Makefile脚本中做集中处理，必须by case去处理。因此添加一个特殊的分区要了解以下步骤。

2.7.1. Kernel分区处理

`image.mk` 中需要写上分区打包的脚本命令。

例如KERNEL分区打包命令：

```
kernel_nofsimage:
    @echo [[${@}]
    cp -rvf $(($(patsubst %_nofsimage,%,${@})$(RESOUCE))
    (IMAGEDIR)/$(patsubst %_nofsimage,%,${@})
```

在 `script_nor.mk/ script _nand.mk` 中写上分区烧录脚本生成的命令。

例如制作Kernel分区烧录脚本的脚本命令：

```
kernel_${FLASH_TYPE}__script:

    @echo "aaaabbbbaaaaaaa ${@}"

    @echo "# <- this is for comment / total file size must be less
    than 4KB" > $(SCRIPTDIR)/[[kernel.es

    @echo tftp $(TFTPDOWNLOADADDR) kernel >>
    $(SCRIPTDIR)/[[kernel.es

    @echo nand erase.part KERNEL >> $(SCRIPTDIR)/[[kernel.es

    @echo nand write.e $(TFTPDOWNLOADADDR) KERNEL \${filesize} >>
```

```

$(SCRIPTDIR)/[[kernel.es

    @echo nand erase.part RECOVERY >> $(SCRIPTDIR)/[[kernel.es

    @echo nand write.e $(TFTPDOWNLOADADDR) RECOVERY \${filesize}
>> $(SCRIPTDIR)/[[kernel.es

    @echo "% <- this is end of file symbol" >>
$(SCRIPTDIR)/[[kernel.es
    @echo kernel-image done!!!

```

2.7.2. 带备份的分区处理

SPINAND的分区中IPL、IPL_CUST、UBOOT分区一般多个备份。

例如IPL在烧录时，每个IPL的data会在每个block开始各占一份，一个block的大小为128kb，一般IPL的data不超过20kb，假设IPL的数据有六个备份，那么整个IPL分区的大小等于6个block，在分区打包的时候用dd命令把IPL分区进行打包，使输出的bin文件能够与IPL分区完全映射，6份IPL的data各自分布在每个block的开头。

设置带备份的分区，参考如下代码：

```

ipl$(RESOUC) = $(PROJ_ROOT)/board/$(CHIP)/boot/ipl/IPL.bin

ipl$(DATASIZE) = 0x20000

ipl$(COPIES) = 6

ipl$(BKCOUNT) = 5

ipl$(PATSIZE) = $(call multiplyhex, $(ipl$(COPIES)),
$(ipl$(DATASIZE)))

ipl$(MTDPART) = $(ipl$(DATASIZE))@$(cis$(CISSIZE))
(IPL0)$(ipl$(BKCOUNT))

```

ipl\$(COPIES) 表示IPL的data会有多少个备份。

ipl\$(DATASIZE) 表示一份data的大小，IPL的bin文件若不超过1个block的大小则按照1个block的大小来算。

ipl\$(PATSIZE) 是IPL实际分区的大小，makefile中做了一个乘法计算，就是备份的个数乘以data大小。

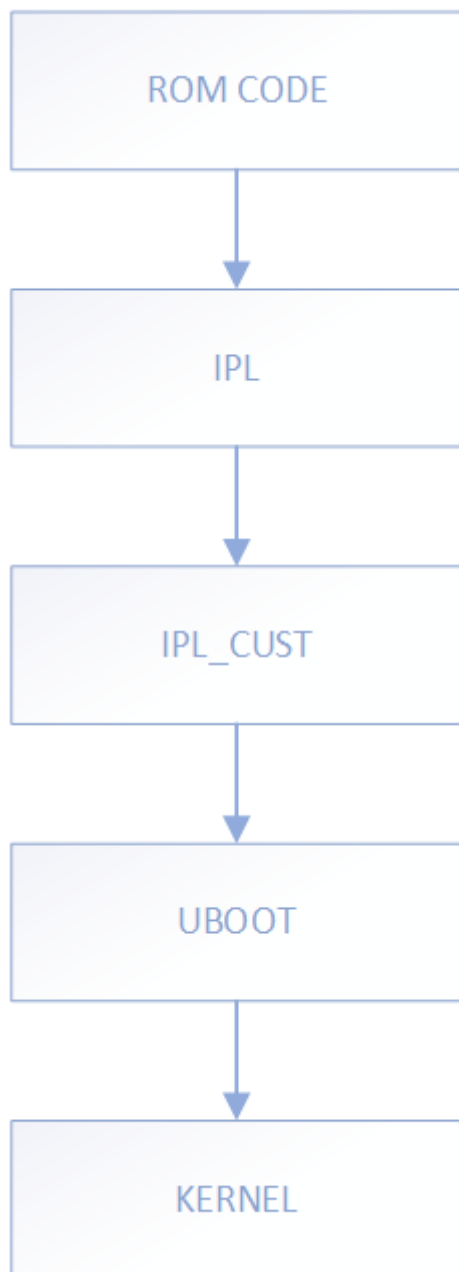
`ipl$(BKCOUNT)` 是PNI数据结构中特有的一种数据，在系统运行时，没有坏块的情况下，系统默认使用的是IPL的第一份data，但若是第一份data所在的block产生坏块，则ROM code会偏移一个block去找IPL，这个BKCOUNT变量就是告诉ROM code可以偏移block的最大个数。

`ipl$(MTDPART)` 是给pnigenerator用作解析分区配置的变量，它会传递给 `cis$(BOOTTAB)` 或者 `cis$(SYSTAB)`，若没有BKCOUNT的情况下，语法格式符合 `uboot mtdparts` 的标准，而在没有定义变量MTDPARTS的情况下，uboot在执行 `write cis` 命令时会解析pni的值，并且转换成mtdparts字串保存在环境变量中。

2.7.3. MXP与PARTINFO

ARM启动后会先进入一段固定的ROM CODE中：

如图所示：



ROM code启动后会按照固定的地址启动到IPL，IPL会先去找一份原始的分区配置 raw data，这部分就是MXP或PARTINFO。

MXP/PARTINFO保存的是SPINOR/SPINAND分区的配置信息，arm启动执行到IPL时还没有mtdpart的概念，所以去读MXP或PARTINFO中的内容才能找到IPL_CUST或者uboot，因此MXP/PARTINFO所在的地址是固定的。

2.8. SPINOR的MXP

SPINOR的分区修改由mxpgenerator通过传入的不同参数动态生成后汇总到MXP_SF.bin。在ALKAID的编译打包环境中image.mk负责分区的打包步骤，通过shell脚本生成MXP_SF.bin。

如图所示：

```
/home/malloc.peng/ALKAID/project/image/makefiletools/bin/mxpgenerator "0x10000(IPL),0x10000(IPL_CUST),0x10000(MXPT),0x1F000(UBOOT),0x1000(UBOOT_ENV)" "0x200000(KERNEL),0x400000(rootfs)"
: [[156]]
BOOT: 0x10000(IPL),0x10000(IPL_CUST),0x10000(MXPT),0x1F000(UBOOT),0x1000(UBOOT_ENV)
SYS: 0x200000(KERNEL),0x400000(rootfs),0x300000(miservice),0x680000(customer)
MXP count max 30
NOR FLASH HAS USED 0x1000000KB
IPL: 0x00000000-0x00010000 size:64KB
IPL_CUST: 0x00010000-0x00020000 size:64KB
MXPT: 0x00020000-0x00030000 size:64KB
UBOOT: 0x00030000-0x0004F000 size:124KB
UBOOT_ENV: 0x0004F000-0x00050000 size:4KB
BOOT: 0x00050000-0x00050000 size:320KB
KERNEL: 0x00050000-0x00250000 size:2048KB
rootfs: 0x00250000-0x00650000 size:4096KB
miservice: 0x00650000-0x00950000 size:3072KB
customer: 0x00950000-0x01000000 size:6848KB
Available MXP record count:10
[[ipl_nofimage]]
```

mxpgenerator执行命令：

```
mxpgenerator
"0x10000(IPL),0x10000(IPL_CUST),0x10000(MXPT),0x1F000(UBOOT),0x1000(UBOOT_ENV)"
"0x200000(KERNEL),0x400000(rootfs),0x300000(miservice),0x680000(customer)"
/home/malloc.peng/ALKAID/project/image/output/images/boot/MXP_SF.bin
```

mxpgenerator有三个参数: mxpgenerator [v0] [v1] [v2]:

- [v0]
传入的是BOOT部分的分区配置
- [v1]
传入的是sys分区的部分
- [v2]
输出文件路径。

mxpgenerator工具的源码路径：

```
project/image/makefiletools/src/mxpgenerator/mxpgenerator.c
```

利用服务器上的gcc编译：

```
gcc mxpgenerator.c -o mxpgenerator
```


2.9. SPINAND的PARTINFO

PARTINFO.pni 通过 **pnigenerator** 生成 **PARTINFO.pni**，保存在CIS分区中，其中包括一些spinand基本的信息。

```
BOOT: 0x140000(CIS),0x20000@0x140000(IPL0)5,0x20000(IPL_CUST0)2,0x20000(IPL_CUST1)2,0x4
SYS: 0x500000(KERNEL),0x500000(RECOVERY),0x600000(rootfs),-(UBI)
FLASH HAS USED 0x8000000KB
ChkSum      : 1043
SpareByteCnt : 64
PageByteCnt  : 2048
BlkPageCnt   : 64
BlkCnt       : 1024
PartCnt      : 11
UnitByteCnt  : 8
Checksum ok!!
IDX:      StartBlk:      BlkCnt:      BackupBlkCnt:      PartType:
0:      0,(0000000000)    10,(0x00140000)    0,(0000000000)    0x0020,(CIS)
1:      10,(0x00140000)    1,(0x00020000)    5,(0x000A0000)    0x0003,(IPL)
2:      16,(0x00200000)    1,(0x00020000)    2,(0x00040000)    0x0001,(IPL_CUST)
3:      19,(0x00260000)    1,(0x00020000)    2,(0x00040000)    0x0001,(IPL_CUST)
4:      22,(0x002C0000)    2,(0x00040000)    1,(0x00020000)    0x0006,(UBOOT)
5:      25,(0x00320000)    2,(0x00040000)    1,(0x00020000)    0x0006,(UBOOT)
6:      28,(0x00380000)    2,(0x00040000)    0,(0000000000)    0x000D,(ENV)
7:      30,(0x003C0000)    40,(0x00500000)    0,(0000000000)    0x0012,(KERNEL)
8:      70,(0x008C0000)    40,(0x00500000)    0,(0000000000)    0x0009,(RECOVERY)
9:      110,(0x00DC0000)    48,(0x00600000)    0,(0000000000)    0x000F,(ROOTFS)
10:     158,(0x013C0000)    866,(0x06C40000)    0,(0000000000)    0x0021,(UBI)
1+0 records in
```

执行命令：

```
pnigenerator -s 64 -p 2048 -b 64 -k 1024 -u 8 -l 0x20000 -t
"0x60000(CIS),0x20000@0x60000(IPL0),0x20000(IPL1),0x20000(IPL2),0x200
-y "0x500000(KERNEL),0x500000(RECOVERY),0x600000(rootfs),-(UBI)" -
o
/home/malloc.peng/ALKAID/project/image/output/images/boot/PARTINFO.pni
- - - - -
```

pnigenerator [options]

Parameter meaning:

-s Spare byte count.

-p Page byte count.

-b Block page count.

-k Block count.

-u Unit byte count.

-l Block size.

-t BOOT Part

-y SYS Part

-r Dump pni file

-o Output file path.

Source path of the pnigenerator tool:

```
project/image/makefiletools/src/pnigenerator/pnigenerator.c
```

Compile with gcc on the server:

```
gcc pnigenerator.c -o pnigenerator
```

It is enough to save the partition information of the BOOT part in PARTINFO.bin, and other partitions can be added to the mtdparts of uboot.